

End to End CI CD pipeline Jenkins, Docker and Kubernetes Integration.

starts at 9:05 pm

Agenda

1. Recap
2. Understanding the Project
3. Jenkins Role in our project
4. Methods to implement our project
5. Understanding Application Code
6. Connecting Jenkins to AWS EKS
7. Installing required plugins
8. Understanding Dockerfile, Deployment and Service file.
9. Creating Pipeline
10. Jenkins Credentials needed in the pipeline
11. Implementing Webhook
12. Doing a version Upgrade

Automation

→ Recap

→ What all have we learnt ?

- ① jobs / pipelines
- ② credentials
- ③ plugins

④ triggers

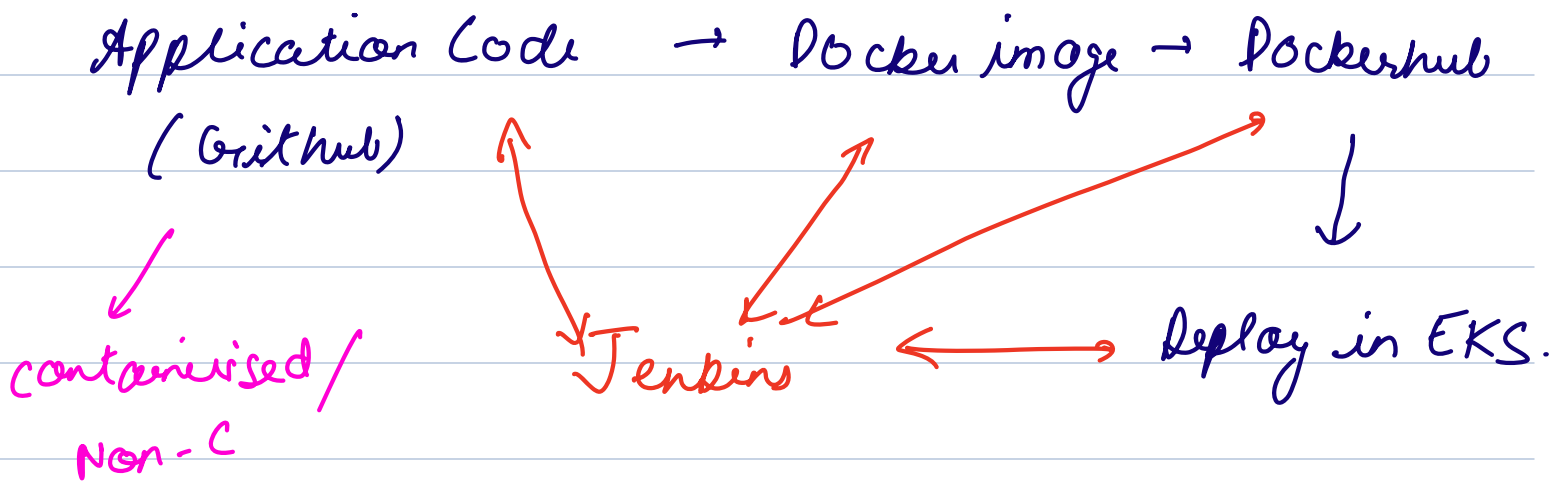
→ webhook

Docker

(Dockerhub)

Kubernetes

→ Understanding the Project



Jenkins can handle **any deployment type** with appropriate:

- **Plugins**.

- **Pipeline configurations**.

→ Role of Jenkins in our Project.

① SCM → Automating Code Fetching.

(Pullin latest code)

② Build Process. + Automate Image creation.

→ Fresh image

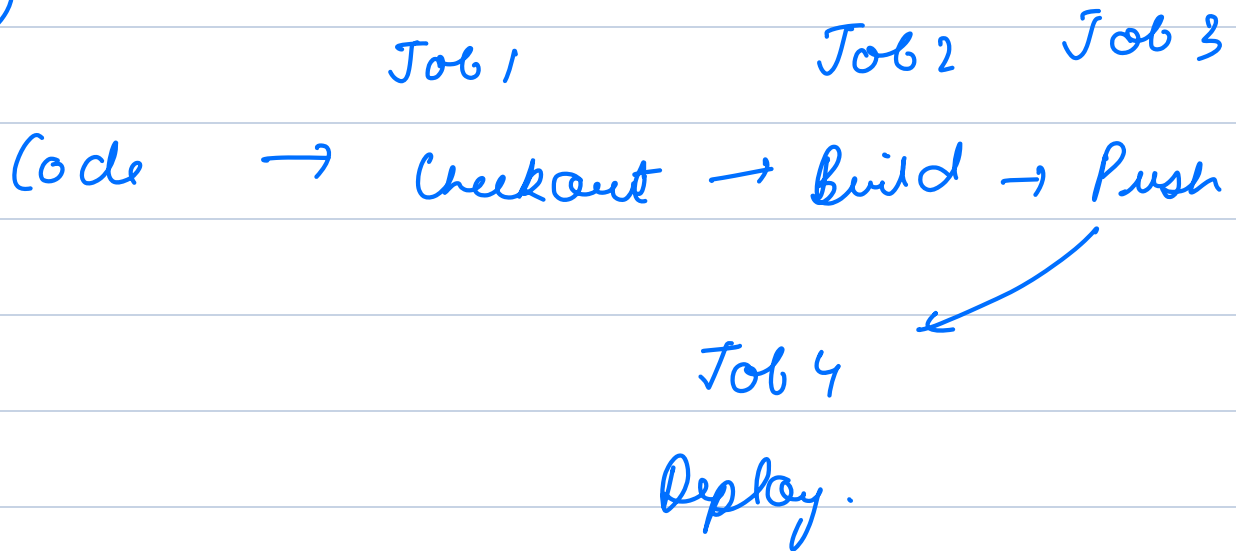
③ Docker Authentication.
(Dockerhub credentials)

④ Image Push.

⑤ Kubernetes Deployment → Automating release to EKS.

→ Methods to implement the Project.

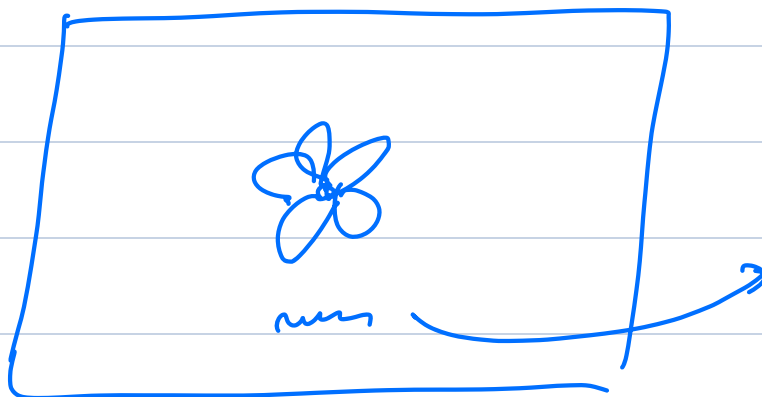
①



② Declarative Pipeline

→ Application Code Logic.

Static Page



Recent
documentation.

→ Connecting Jenkins Server to AWS EKS.
Prerequisites

① → AWS CLI

② → AWS Credentials

→ Access Key.

→ Secret Access Key

③ Docker installed

④ Kubectl

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
```

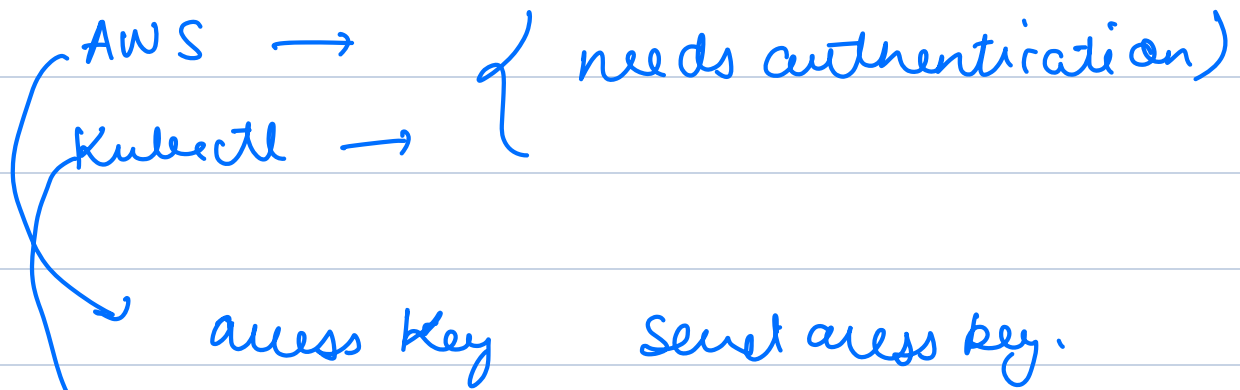
```
unzip awscliv2.zip
```

```
sudo ./aws/install
```

→ authenticate with AWS

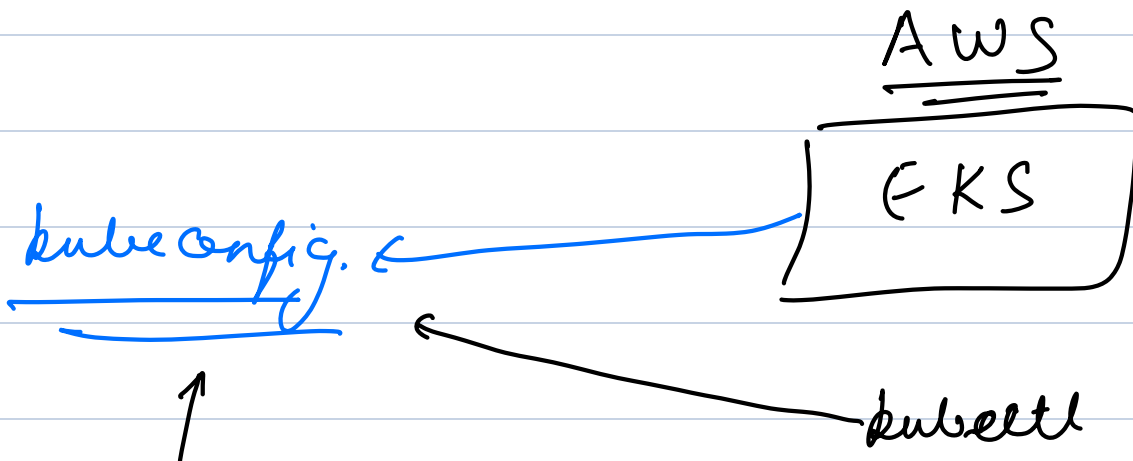
→ aws configure

Add EKS cluster to kubeconfig



vi ~/.aws/credentials

kubeconfig.



aws eks --region us-east-1 update-kubeconfig --name jenkinsProject

→ gimme aws creds.

sudo su jenkins

aws configure

→ Plugins needed for the pipeline

- Docker Plugin

- Docker API Plugin

- Kubernetes Plugin

- Kubernetes API Plugin

- Kubernetes CLI Plugin

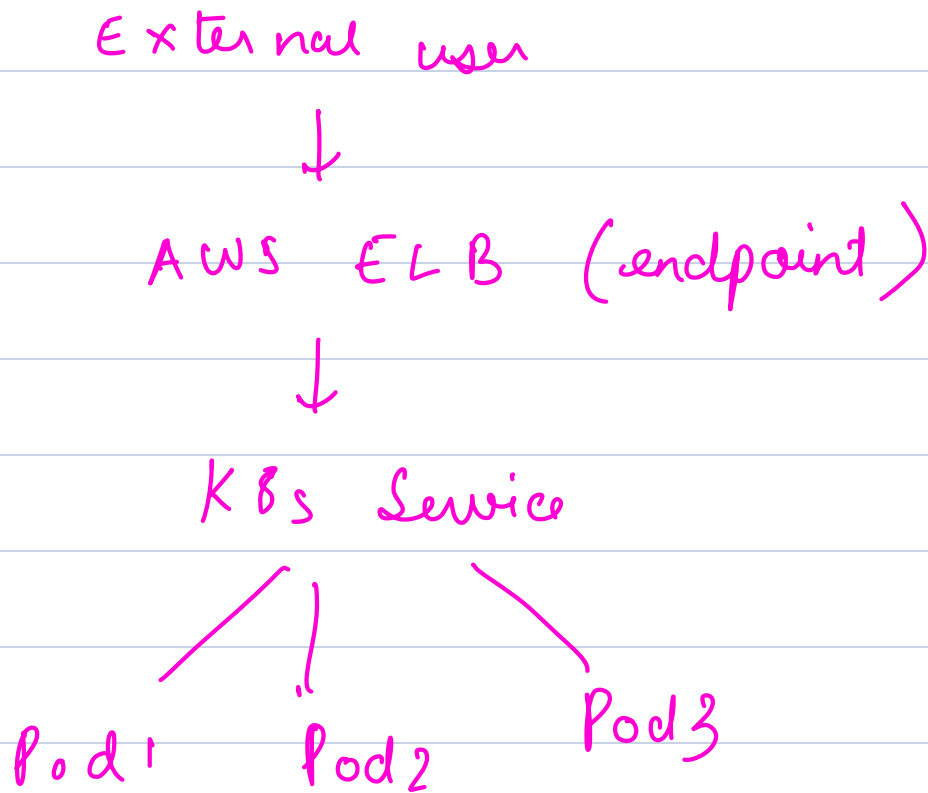
- Kubernetes Credentials Plugin

Break → 10:15 pm

→ Understanding Dockerfile

This Dockerfile builds a lightweight React.js container that:

1. Uses **Node.js 23.5.0 Alpine** as the base image.
2. Copies necessary files (`package.json`, `package-lock.json`).
3. Installs dependencies.
4. Copies the full application source code into the container.
5. Exposes port **3000** for access.
6. Runs `npm start` to launch the React development server.



→ Jenkins Pipeline

1. **Checking out the source code** from GitHub
2. **Building a Docker image** for the React app
3. **Authenticating with Docker Hub**
4. **Pushing the image** to Docker Hub
5. **Deploying the app** on an **EKS cluster**